Bases de données & Réseaux

Comprendre où sont les données et comment les composants communiquent

PARTIE 1

Bases de données

Persistance, cohérence, disponibilité et performance

Comprendre le principe avant les outils

Pourquoi une base de données ?

Une application doit conserver, organiser et protéger ses données.



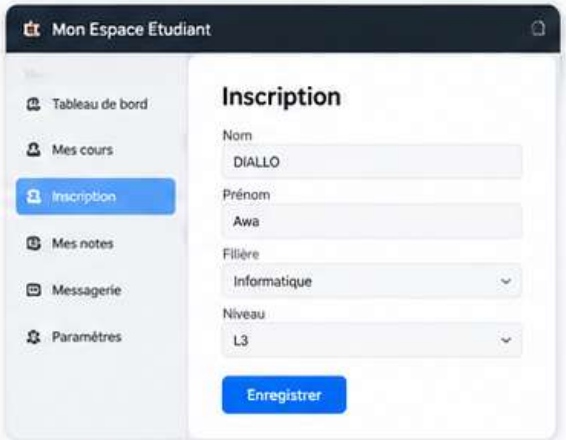
Les données d'aujourd'hui, les services de demain.

- Une application doit conserver les informations au-delà de l'exécution.
- La base de données structure les données pour les retrouver, les modifier et les protéger.
- Le SGBD devient l'intermédiaire entre l'application et les données persistantes.
- Le choix de la base dépend du type de données et des exigences métier.

1. Utilisateur
Interagit avec l'application



2. Application
Envoie et reçoit des données



3. SGBD
Gère les données

Exemples de SGBD

PostgreSQL · MySQL · Microsoft SQL Server · ORACLE · MongoDB · Redis

4. Données
Stockées de manière persistante

Étudiants | Cours | Inscriptions

id	nom	prenom	filiere	niveau
1	Diallo	Awa	Informatique	L3
2	Fall	Mouhamadou	Réseaux	M2
3	Ndiaye	Fatou	Gestion	L1
4	Ba	Cheikh	Informatique	L2
5	Sarr	Mariama	Réseaux	L3

5 enregistrements

En résumé

Une base de données permet de stocker, organiser, retrouver et protéger les données de l'application de manière fiable et durable.

Intégrité des données · Accès rapide · Multi-utilisateurs · Sauvegarde · Sécurité

Donnée, base et SGBD

Des concepts complémentaires au cœur de l'architecture applicative.



Les données d'aujourd'hui, les services de demain.

À retenir

Définitions et rôle dans l'architecture

1. **Une donnée** est une information manipulée par le système.
2. **Une base de données** est un ensemble organisé et persistant.
3. **Un SGBD** gère le stockage, les requêtes, les droits, les transactions et la récupération.
4. Dans une architecture, **le SGBD** constitue une brique technique critique.

Donnée	Information élémentaire (manipulée par le système).
Base de données	Ensemble organisé et persistant.
SGBD	Gère le stockage, les requêtes, les droits, les transactions et la récupération.
Application	Utilise les données via le SGBD (pour offrir des services).

Le SGBD fait le lien entre l'application et les données persistantes.

COMPARER

Fichier ou base de données ?

Deux approches pour stocker des informations, avec des usages et des limites différents.

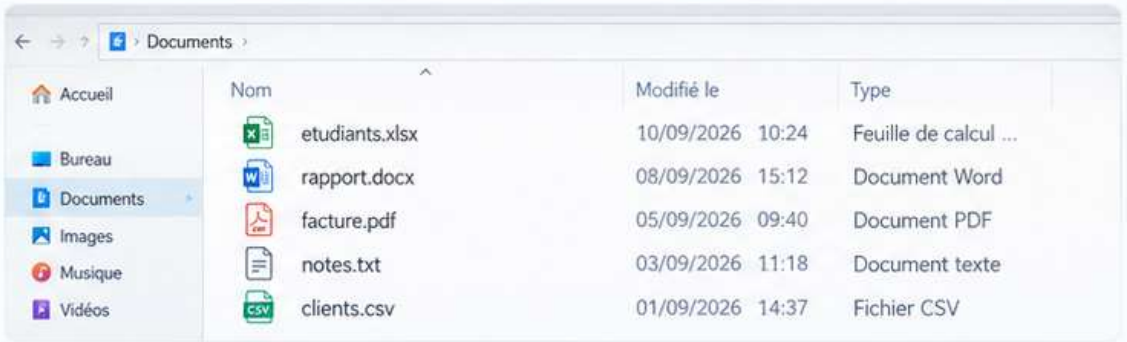


Choisir la bonne solution selon vos besoins.

Fichier

Une méthode simple pour stocker des données.

- Stockage simple (un fichier sur le disque)
- Peu de contrôle sur les accès concurrents
- Recherche limitée
- Risque d'incohérence si plusieurs utilisateurs modifient les données
- Adapté aux petits volumes et aux usages personnels



Exemples d'utilisation

Excel · Word · Bloc-notes · Fichier CSV · PDF · Fichier texte

Base de données

Une solution structurée et robuste pour les applications.

- Données structurées et organisées (tables, relations)
- Contraintes, droits et transactions
- Requêtes puissantes et adaptées
- Multi-utilisateurs avec gestion de la concurrence
- Meilleure cohérence et intégrité des données
- Adaptée aux applications métier et aux grands volumes



Exemples de SGBD

PostgreSQL · MySQL · Microsoft SQL Server · ORACLE · MongoDB · SQLite

À retenir

- Utilisez un fichier pour des besoins simples et personnels.
- Utilisez une base de données pour des applications collaboratives et critiques.
- Le choix dépend du volume de données, du nombre d'utilisateurs et des exigences métier.

Objectif

Choisir la bonne solution de stockage pour un système d'information fiable et évolutif.

Rôle architectural du SGBD

Le SGBD est le cœur de la gestion des données dans un système d'information.



Des données fiables pour des décisions éclairées.

Utilisateurs / Applications

- **Application Web** (ex. Moodle, e-learning)
- **Application de bureau** (ex. Microsoft Excel)
- **Application mobile** (ex. application métier)
- **Applications métiers** (ex. ERP, Scolarité, RH)

Requêtes SQL (SELECT, INSERT, ...)

Résultats (données)

SGBD



PostgreSQL

MySQL

Microsoft SQL Server

ORACLE

MongoDB

Gestion des données

Stockage • Sécurité • Cohérence • Disponibilité

Lecture / Écriture

Données persistantes



Bases de données

Fichiers

Stockage NAS / SAN

Principales responsabilités

1. **Stocker** — Enregistrer les données de manière organisée et durable.
2. **Rechercher** — Permettre des recherches rapides et efficaces.
3. **Modifier** — Gérer les insertions, mises à jour et suppressions.
4. **Sécuriser** — Contrôler les accès, garantir l'intégrité et la confidentialité.
5. **Restaurer** — Assurer la sauvegarde et la restauration des données.

À retenir

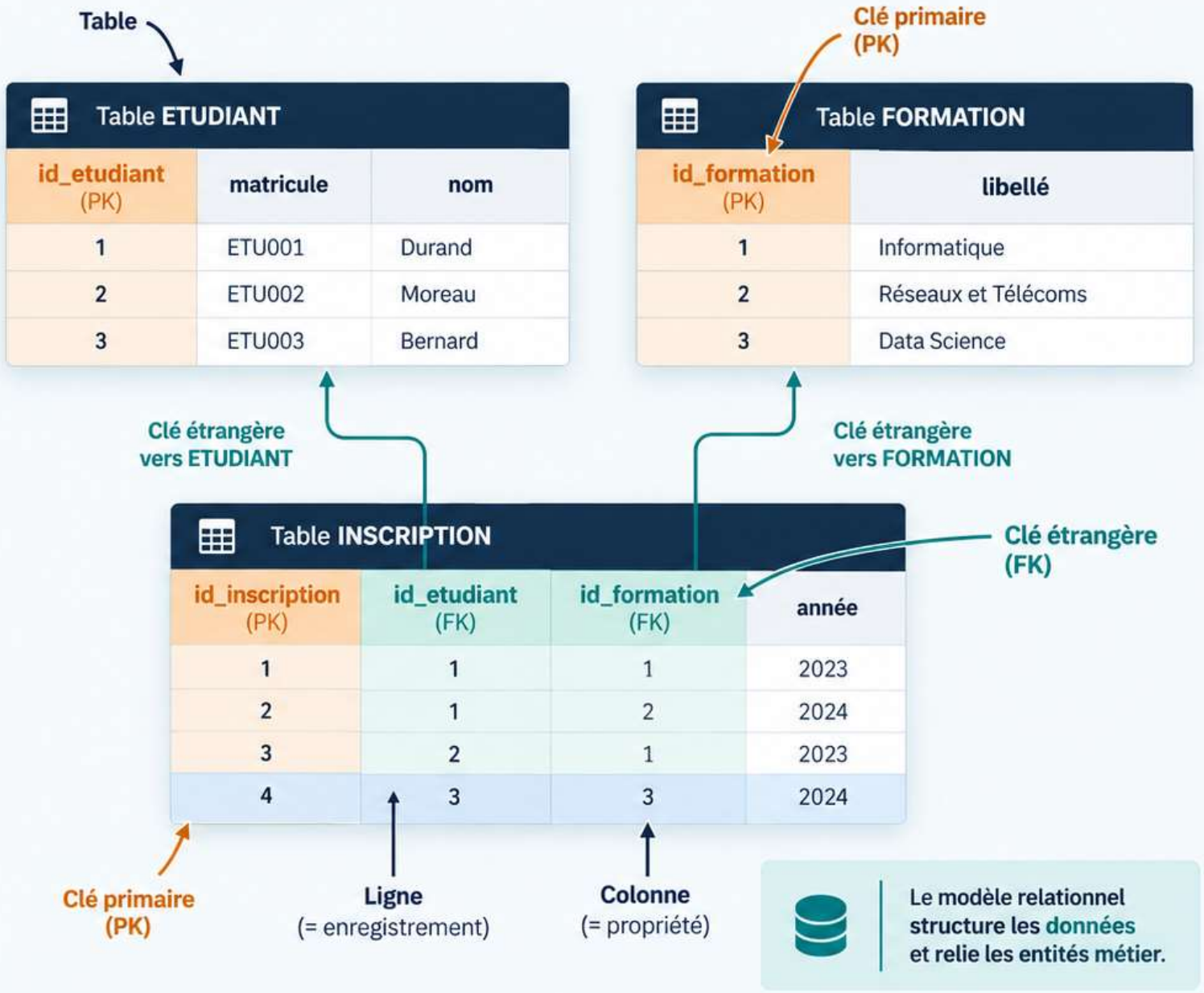
- Le SGBD protège les données et contrôle leur accès.
- Il garantit des règles (unicité, relations, cohérence).
- Il permet plusieurs accès simultanés.
- Il constitue souvent un point critique de disponibilité.

Bonnes pratiques

- Choisir un SGBD adapté aux besoins.
- Mettre en place des sauvegardes régulières.
- Sécuriser les accès (authentification, rôles).
- Surveiller les performances et la disponibilité.

Modèle relationnel

- Les données sont organisées en tables.
- Chaque ligne représente un enregistrement.
- Chaque colonne décrit une propriété.
- Les tables sont reliées par des clés primaires et étrangères.



SYNTHÈSE

Exemple relationnel universitaire

Un modèle simple pour gérer les étudiants, les formations, les inscriptions et les paiements.



Des données organisées pour un monde réel.

ETUDIANT

id	INT
matricule	VARCHAR(20)
nom	VARCHAR(50)
email	VARCHAR(100)
date_naissance	DATE

INSCRIPTION

id	INT
etudiant_id	INT
formation_id	INT
annee	VARCHAR(9)
date_inscription	DATE

FORMATION

id	INT
nom	VARCHAR(100)
niveau	VARCHAR(50)
description	TEXT

PAIEMENT

id	INT
inscription_id	INT
montant	DECIMAL(10,2)
statut	VARCHAR(20)
date_paiement	DATE

Légende

- Clé primaire
- Clé étrangère
- 1 — Relation un-à-plusieurs (1..N)
- N — Plusieurs-à-un (N..1)

Quelques enregistrements (exemples)

id	matricule	nom	email
1	ETU001	Diallo Awa	awa.diallo@ucad.sn
2	ETU002	Fall Mouhamadou	m.fall@ucad.sn
3	ETU003	Ndiaye Fatou	f.ndiaye@ucad.sn
...			

Rôles des tables

- **ETUDIANT** : Identifie les étudiants
- **FORMATION** : Décrit l'offre de formation
- **INSCRIPTION** : Relie un étudiant à une formation
- **PAIEMENT** : Enregistre les règlements

À retenir

- Les données sont organisées en tables.
- Chaque ligne représente un enregistrement.
- Les tables sont reliées par des clés primaires et étrangères.
- Ce modèle facilite la gestion et l'intégrité des données.

Avantages

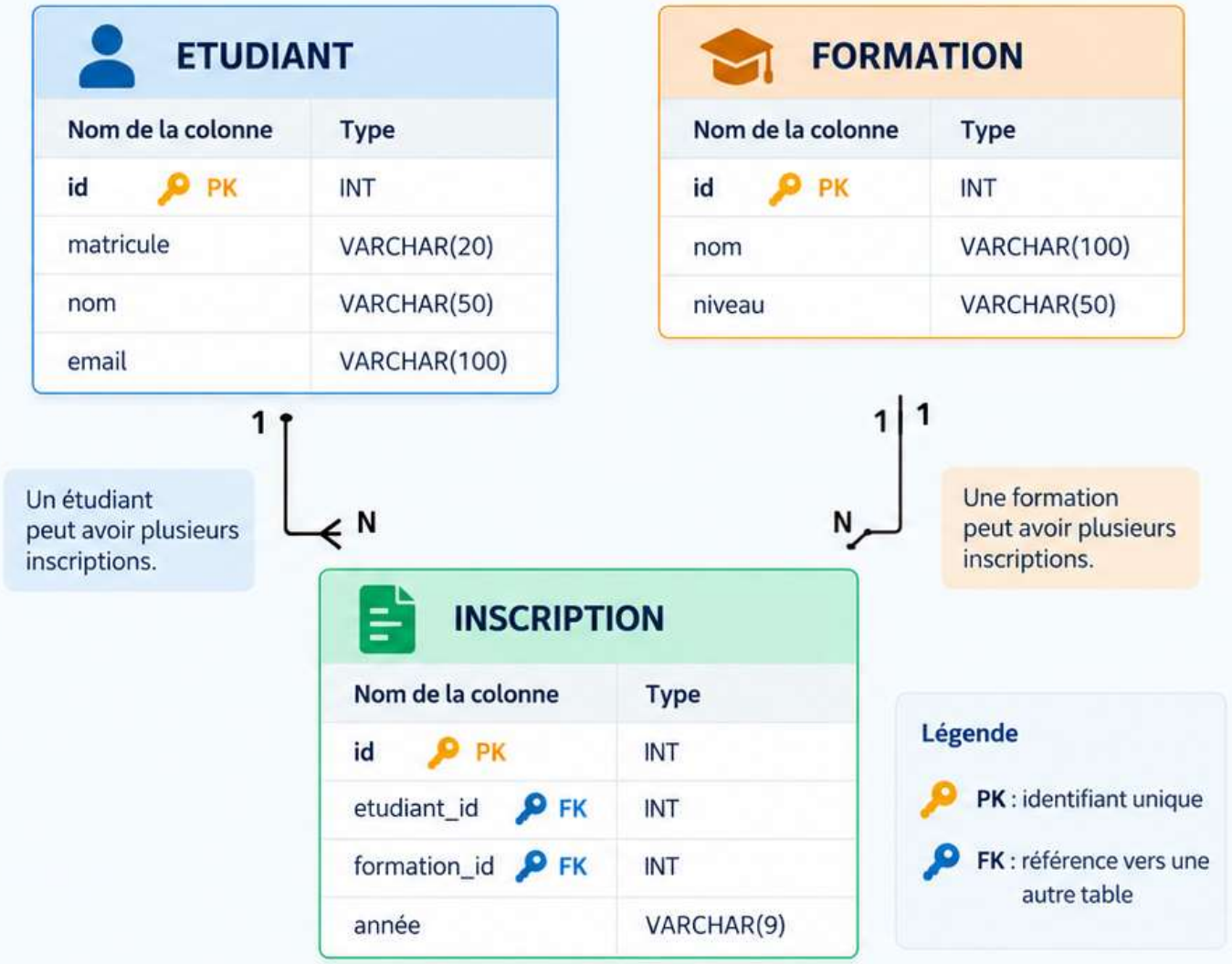
- Données cohérentes et sans duplication
- Requêtes efficaces
- Facile à maintenir et à faire évoluer

Clés et relations

- La clé primaire identifie une ligne de manière unique.
- La clé étrangère référence une autre table.
- La relation évite de dupliquer les mêmes informations.
- Une architecture de données propre facilite la maintenance.

Bon à savoir

Les relations permettent de lier les données sans les dupliquer, pour une base cohérente et évolutive.



SYNTHÈSE

Cardinalités essentielles

Les cardinalités définissent le nombre d'occurrences d'une entité pouvant être liées à une autre.



Des relations claires pour des données cohérentes.

Notation	Lecture métier	Exemple (schéma relationnel)	Exemple de données
1,1	**Obligatoire et unique** Une entité est liée à une et une seule occurrence de l'autre entité.	PERSONNE (id (PK)) — 1 — 1 — PROFIL (id (PK), personne_id (FK), ...)	Une personne possède un seul profil. PERSONNE: id | nom | ... : 1 | Diallo Awa | ... PROFIL: id | personne_id | type : 1 | 1 | Étudiant
0,1	**Optionnel et unique** Une entité peut être liée à zéro ou une seule occurrence de l'autre entité.	UTILISATEUR (id (PK)) — 1 — 0..1 — PHOTO_PROFIL (id (PK), utilisateur_id (FK), ...)	Un utilisateur peut avoir une photo de profil (ou pas). UTILISATEUR: id | nom | ... : 1 | Fall Mouhamadou | ... PHOTO_PROFIL: id | utilisateur_id | url : 1 | 1 | photo.jpg
1,N	**Au moins un, plusieurs possibles** Une entité est liée à une ou plusieurs occurrences de l'autre entité.	FORMATION (id (PK)) — 1 — N — INSCRIPTION (id (PK), formation_id (FK), ...)	Une formation possède plusieurs inscriptions. FORMATION: id | nom : 1 | Licence Informatique ; 2 | Master Réseaux INSCRIPTION: id | formation_id | etudiant_id : 1 | 1 | 10 ; 2 | 1 | 25
0,N	**Optionnel, plusieurs possibles** Une entité peut être liée à zéro, une ou plusieurs occurrences de l'autre entité.	ÉTUDIANT (id (PK)) — 1 — 0..N — INSCRIPTION (id (PK), etudiant_id (FK), ...)	Un étudiant peut n'avoir aucune ou plusieurs inscriptions. ÉTUDIANT: id | nom : 10 | Ndiaye Fatou ; 25 | Ba Cheikh INSCRIPTION: id | etudiant_id | formation_id : 1 | 10 | 1 ; 2 | 10 | 2

À retenir

- La clé primaire identifie une ligne de manière unique.
- La clé étrangère référence une autre table.
- La relation évite de dupliquer les mêmes informations.
- Une architecture de données propre facilite la maintenance.

Bonnes pratiques

- Définir les bonnes cardinalités.
- Utiliser des clés primaires et étrangères.
- Respecter l'intégrité référentielle.
- Documenter le modèle.

AVANT LA BASE

Modélisation des données

Du besoin métier à une base de données *cohérente et durable.*



Une bonne modélisation aujourd'hui, une base fiable demain.

1 Besoin
Comprendre le métier

Exemple :
L'université veut gérer ses étudiants, ses formations, les inscriptions et les paiements.

- Identifier les objectifs
- Recueillir les exigences
- Définir le périmètre

2 Entités
Identifier les objets métier

- Étudiant
- Formation
- Inscription
- Paiement

- Repérer les entités principales
- Nommer clairement
- Définir leur rôle

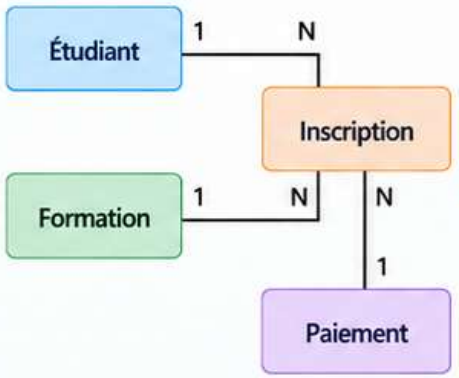
3 Attributs
Décrire les informations

ÉTUDIANT

Attribut	Type
id	INT
matricule	VARCHAR(20)
nom	VARCHAR(50)
email	VARCHAR(100)
date_naissance	DATE
...	...

- Définir les attributs
- Choisir les types de données
- Identifier les clés

4 Relations
Définir les liens et cardinalités



- Définir les cardinalités
- Poser les clés primaires et étrangères
- Éviter les redondances

5 Tables
Obtenir le modèle logique

ETUDIANT

id	matricule	nom	email
1	ETU001	Diallo Awa	awa@ucad.edu.sn
2	ETU002	Fall Mouhamadou	m.fall@ucad.edu.sn
3	ETU003	Ndiaye Fatou	f.ndiaye@ucad.edu.sn
...	...	...	...

INSCRIPTION

id	etudiant_id	formation_id	annee
1	1	1	2024
2	2	1	2024
3	1	2	2025
...	...	...	...

- Traduire en tables relationnelles
- Valider la cohérence du modèle

À retenir

- La modélisation part du besoin métier.
- Elle identifie les entités, attributs, relations et cardinalités.
- Elle évite de créer une base selon l'écran de saisie uniquement.
- Elle prépare une base cohérente et durable.

Bonnes pratiques

- Impliquer les utilisateurs métier.
- Utiliser des conventions de nommage claires.
- Vérifier les cardinalités.
- Documenter le modèle (MCD/MLD).

COMPARER

Bonne et mauvaise modélisation

Une bonne modélisation simplifie la gestion des données et évite les erreurs.

Table unique mal structurée

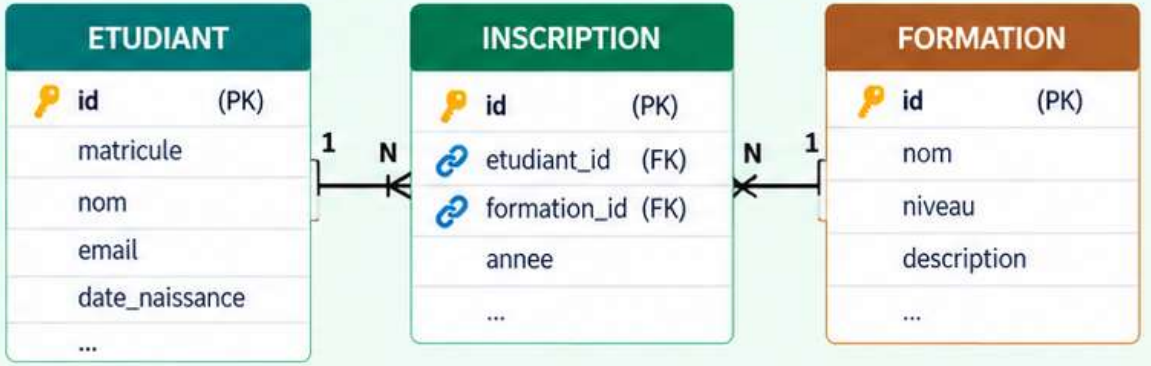
ETUDIANTS_FORMATIONS

id	nom	email	formation1	formation2	formation3
1	Diallo Awa	awa@ucad.edu.sn	L1 Info	L2 Réseaux	-
2	Fall Mouhamadou	m.fall@ucad.edu.sn	L1 Info	-	-
3	Ndiaye Fatou	f.ndiaye@ucad.edu.sn	L1 Info	L2 Réseaux	L3 Systèmes
...	...	...	...	...	...

Problèmes

- Répétitions nombreuses
- Colonnes Formation1, Formation2, Formation3
- Mise à jour difficile
- Risque d'informations contradictoires

Tables liées



Avantages

- Étudiant, Formation, Inscription séparés
- Relations explicites
- Données plus propres
- Évolutions plus simples
- Meilleure cohérence et intégrité

À retenir

- Éviter les tables trop larges ou mal structurées.
- Utiliser des tables distinctes et des relations claires.
- Une bonne modélisation facilite la maintenance et l'évolution du système.

Objectif

Concevoir des bases de données simples, cohérentes et durables.

Normalisation : principe utile

La normalisation organise les données pour éviter les redondances et garantir leur cohérence.



Des données bien structurées pour un système plus fiable.

Données non normalisées

Une seule table avec des informations répétées.

ETUDIANTS_FORMATIONS (non normalisée)

id	nom	email	formation1	formation2	formation3
1	Diallo Awa	awa@ucad.edu.sn	L1 Info	L2 Réseaux	-
2	Fall Mouhamadou	m.fall@ucad.edu.sn	L1 Info	-	-
3	Ndiaye Fatou	f.ndiaye@ucad.edu.sn	L1 Info	L2 Réseaux	L3 Systèmes
...	...	...	...	...	...

Problèmes

- Répétitions nombreuses
- Colonnes Formation1, Formation2, Formation3
- Mise à jour difficile
- Risque d'informations contradictoires

Normalisation
(décomposition des données)

Données normalisées

Des tables liées, sans redondance.

ETUDIANT

id (PK)	matricule	nom	email
1	ETU001	Diallo Awa	awa@ucad.edu.sn
2	ETU002	Fall Mouhamadou	m.fall@ucad.edu.sn
3	ETU003	Ndiaye Fatou	f.ndiaye@ucad.edu.sn
...	...	...	...

FORMATION

id (PK)	nom	niveau
1	Licence Info	L1
2	Master Réseaux	L2
3	Licence Systèmes	L3
...	...	...

1 — N

1 — N

INSCRIPTION

id (PK)	etudiant_id (FK)	formation_id (FK)	année
1	1	1	2024
2	1	2	2024
3	3	3	2025
...	...	...	...

Avantages

- Étudiant, Formation, Inscription séparés
- Relations explicites
- Données plus propres
- Évolutions plus simples
- Meilleure cohérence et intégrité

À retenir

- La normalisation réduit les duplications.
- Elle sépare les informations selon leur responsabilité.
- Elle limite les anomalies de modification et suppression.
- Le but est de comprendre la séparation, pas de mémoriser toutes les formes normales.

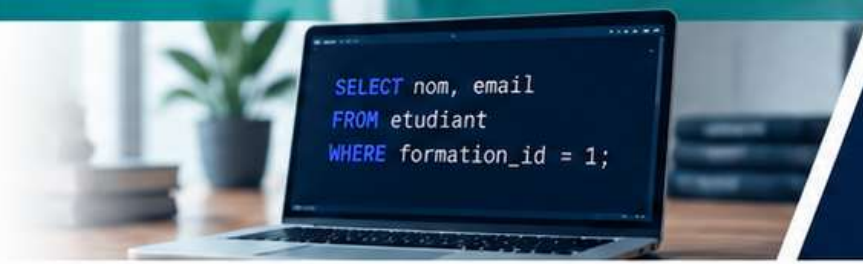
Bonnes pratiques

- Commencer par le besoin métier.
- Utiliser des entités claires et des relations explicites.
- Vérifier l'intégrité des données.
- Documenter le modèle.

LANGAGE DE REQUÊTE

SQL dans l'architecture

SQL est le langage qui fait le lien entre l'application et les données.



Des requêtes claires pour des données fiables.

Application
Interface utilisateur

- Saisie des données
- Consultation des informations
- Modification et suppression
- Fonctionnalités métier

L'application envoie des requêtes SQL via une couche d'accès aux données.

Couche données
Accès et gestion des données

- Gestion des connexions
- Exécution des requêtes SQL
- Contrôle des droits
- Gestion des transactions
- Traduction des résultats

Cette couche assure la sécurité, la cohérence et l'optimisation des accès.

SQL
Langage de requête

```sql
SELECT e.id, e.nom, e.email
FROM etudiant e
JOIN inscription i
ON e.id = i.etudiant_id
WHERE i.formation_id = 1;
```

- Lire (SELECT)
- Insérer (INSERT)
- Modifier (UPDATE)
- Supprimer (DELETE)
- Contrôler les droits (GRANT, REVOKE)

SGBD
Stockage et gestion

- Stocke les données
- Assure l'intégrité
- Gère les relations
- Optimise l'exécution
- Gère la concurrence
- Permet la sauvegarde

Le SGBD exécute les requêtes et renvoie les résultats à l'application.

Exemple de requête

```sql
SELECT e.id, e.nom, e.email, f.nom AS formation
FROM etudiant e
JOIN inscription i ON e.id = i.etudiant_id
JOIN formation f ON i.formation_id = f.id
WHERE f.niveau = 'L1'
```

Résultat

id	nom	email	formation
1	Diallo Awa	awa@ucad.edu.sn	Licence Info
2	Fall Mouhamadou	m.fall@ucad.edu.sn	Licence Info
3	Ndiaye Fatou	f.ndiaye@ucad.edu.sn	Licence Info
...	...	...	...

Points clés

- SQL sert à lire et modifier les données relationnelles.
- Une application utilise SQL directement ou via une couche d'accès aux données.
- Les requêtes doivent respecter les droits et la cohérence métier.
- SQL n'est pas seulement un langage : c'est un contrat d'accès aux données.

À retenir

- SQL est le lien entre l'application et la base de données.
- Il s'intègre dans une architecture en couches.
- La sécurité, la cohérence et les performances sont essentielles.
- Des requêtes bien conçues garantissent des données fiables.

Bonnes pratiques

- Utiliser des requêtes claires et lisibles.
- Respecter le principe du moindre privilège.
- Valider et tester les requêtes.
- Documenter les accès et les usages.

Jointures

La jointure permet de combiner les données de plusieurs tables grâce à leurs clés.

- Les informations utiles sont souvent réparties dans plusieurs tables.
- La jointure rassemble les données liées par des clés (primaires et étrangères).
- Elle exprime la force principale du modèle relationnel.
- Elle permet de lire le métier à travers les relations.

Exemple de requête

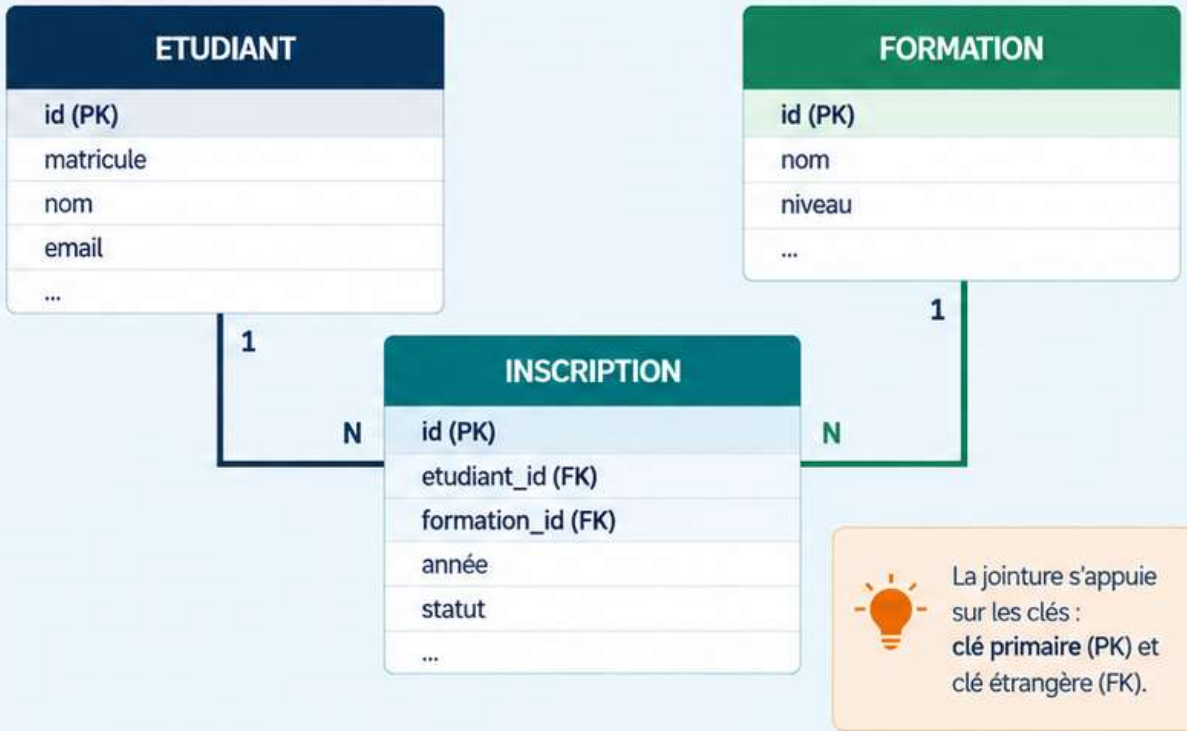
```sql
SELECT e.id, e.nom, e.email, f.nom AS formation
FROM etudiant e
JOIN inscription i ON e.id = i.etudiant_id
JOIN formation f ON i.formation_id = f.id
WHERE f.niveau = 'L1';
```

Résultat attendu

id	nom	email	formation
1	Diallo Awa	awa@ucad.edu.sn	Licence Info
2	Fall Mouhamadou	m.fall@ucad.edu.sn	Licence Info
3	Ndiaye Fatou	f.ndiaye@ucad.edu.sn	Licence Réseaux
...	...	...	...

Jointure dans le modèle relationnel

Exemple : obtenir les étudiants et leur formation via la table d'inscription.



La jointure s'appuie sur les clés : **clé primaire (PK)** et clé étrangère (FK).

À retenir

- Les données sont réparties dans plusieurs tables.
- La jointure utilise les clés pour relier les tables.
- Elle permet d'obtenir une vue métier complète et cohérente.

Bonnes pratiques

- Utiliser des alias pour plus de lisibilité.
- Vérifier les cardinalités et l'intégrité des clés.
- Filtrer les résultats avec WHERE.
- Tester les requêtes sur des jeux de données réels.

PARTIE 2

SQL et NoSQL

Choisir une technologie selon la nature des données

Comprendre le principe avant les outils

COMPARER

SQL et NoSQL : éviter le faux débat

Deux approches complémentaires adaptées à des besoins différents.

Il ne s'agit pas de choisir un camp, mais d'utiliser le bon outil au bon moment.

SQL

Pour des données structurées et cohérentes

Données structurées
Modèle relationnel avec des tables et un schéma fixe.

Relations importantes
Intégrité des données avec des clés et des contraintes.

Transactions fortes
Garantit la cohérence (ACID).

Domaines d'application
Gestion, finance, scolarité, ERP, systèmes critiques.

Exemples : PostgreSQL, MySQL, Oracle, SQL Server

VS

NoSQL

Pour des données flexibles et à grande échelle

Schéma plus flexible
Structure adaptable, pas toujours de schéma fixe.

Volumes ou formats variables
Gère facilement de grandes quantités de données diverses (texte, image, JSON, etc.).

Modèles variés
Document, clé-valeur, colonne, graphe... selon le besoin.

Cas d'usage
Logs, sessions, événements, cache, applications web et temps réel.

Exemples : MongoDB, Cassandra, Redis, Elasticsearch

À retenir

- SQL et NoSQL répondent à des besoins différents.
- SQL privilégie la cohérence et les relations.
- NoSQL privilégie la flexibilité et la scalabilité.

Bonne pratique

- Analyser les besoins du projet.
- Choisir l'outil adapté au cas d'usage.
- Éviter les choix idéologiques : le contexte prime.

SYNTHÈSE

Familles NoSQL

Plusieurs modèles pour des besoins différents.

Il n'existe pas un seul NoSQL, mais plusieurs familles adaptées à des cas d'usage spécifiques.

Famille	Exemple	Usage typique	Particularités
Document Stocke des documents de type JSON.	MongoDB	• Documents JSON • Données semi-structurées • Applications web, catalogues, CMS	✓ Schéma flexible ✓ Facile à faire évoluer ✓ Idéal pour des données variées
Clé-valeur Stocke des paires clé → valeur.	Redis	• Cache • Sessions • Données rapides d'accès	✓ Très haute performance ✓ Simplicité du modèle ✓ Adapté aux grands volumes de requêtes
Colonnes Stocke des données en colonnes (pour de grands volumes).	Cassandra	• Très grands volumes distribués • Données de séries temporelles • Applications à haute disponibilité	✓ Très scalable ✓ Conçu pour le traitement de masse ✓ Bien adapté aux écritures fréquentes
Graphe Modélise des relations entre entités.	Neo4j	• Relations complexes • Recommandations • Détection de fraude • Analyse de réseaux sociaux	✓ Optimisé pour les relations ✓ Parcours de graphes rapides ✓ Idéal pour les données fortement connectées

À retenir

- ✓ Chaque famille NoSQL répond à un besoin précis.
- ✓ Le choix dépend du type de données et des usages.
- ✓ Il est possible de combiner plusieurs technologies NoSQL dans un même projet.

Bonnes pratiques

- ✓ Analyser les besoins fonctionnels et non fonctionnels.
- ✓ Prendre en compte la scalabilité et la performance.
- ✓ Évaluer l'écosystème et la communauté.
- ✓ Tester sur un cas d'usage réel.

CRITÈRES

Choisir SQL

SQL est adapté lorsque les données sont structurées et critiques.

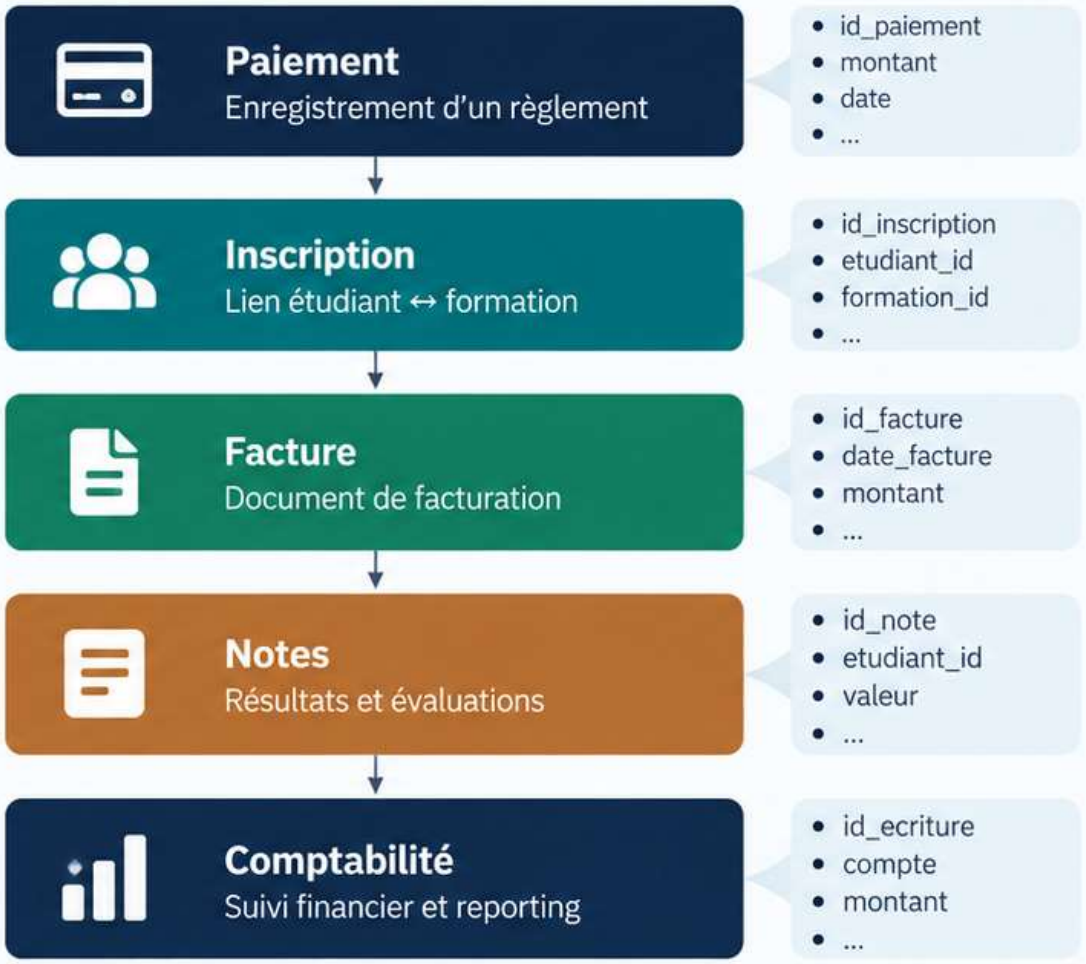
Structure stable
Les données ont une structure claire et bien définie.

Relations importantes
Les objets sont liés par des clés et des contraintes.

Transactions fortes
Les opérations doivent être fiables (ACID).

Coherence élevée
Les opérations métier exigent une cohérence et une intégrité des données.

Usages métiers
Idéal pour la gestion, la finance, la comptabilité, la scolarité, les ERP...



À retenir

- SQL est adapté aux données structurées.
- Il garantit l'intégrité et la cohérence des données.
- Il est idéal pour les processus métier critiques.

Bonnes pratiques

- Analyser les besoins métier.
- Vérifier les relations et les contraintes.
- Prévoir l'évolutivité et la performance.

CRITÈRES

Choisir NoSQL

NoSQL est adapté lorsque les données sont variables, volumineuses et moins structurées.

Données hétérogènes
Les données changent beaucoup de format (texte, image, JSON, vidéo, etc.).

Très grands volumes
Les volumes ou débits de données sont très importants.

Usages orientés événements
Le besoin est orienté document, événement, session ou graphe.

Flexibilité du schéma
Le schéma est plus flexible et peut évoluer facilement.

Approche hybride possible
Une même architecture peut combiner SQL et NoSQL selon les besoins.

Exemples d'usages NoSQL

Logs — Journalisation et traçabilité
- Logs applicatifs
- Logs système
- Logs d'audit

Sessions — Gestion des sessions utilisateur
- Sessions web
- Panier d'achat
- Données temporaires

Événements — Flux temps réel
- Événements IoT
- Notifications
- Streaming de données

Catalogue — Contenu et données variées
- Produits, médias
- Documents, JSON
- Recherche et indexation

Relations complexes — Analyse de relations
- Réseaux sociaux
- Recommandations
- Détection de fraude

À retenir

- NoSQL est adapté aux données variées et volumineuses.
- Il offre une grande flexibilité de schéma.
- Il peut compléter SQL dans une architecture hybride.

Bonnes pratiques

- Analyser la nature des données et les usages.
- Choisir la bonne famille NoSQL (document, clé-valeur, colonnes, graphe).
- Évaluer la scalabilité et les contraintes opérationnelles.

COHÉRENCE MÉTIER

Transactions

Garantir la cohérence des données, même en cas d'erreur.

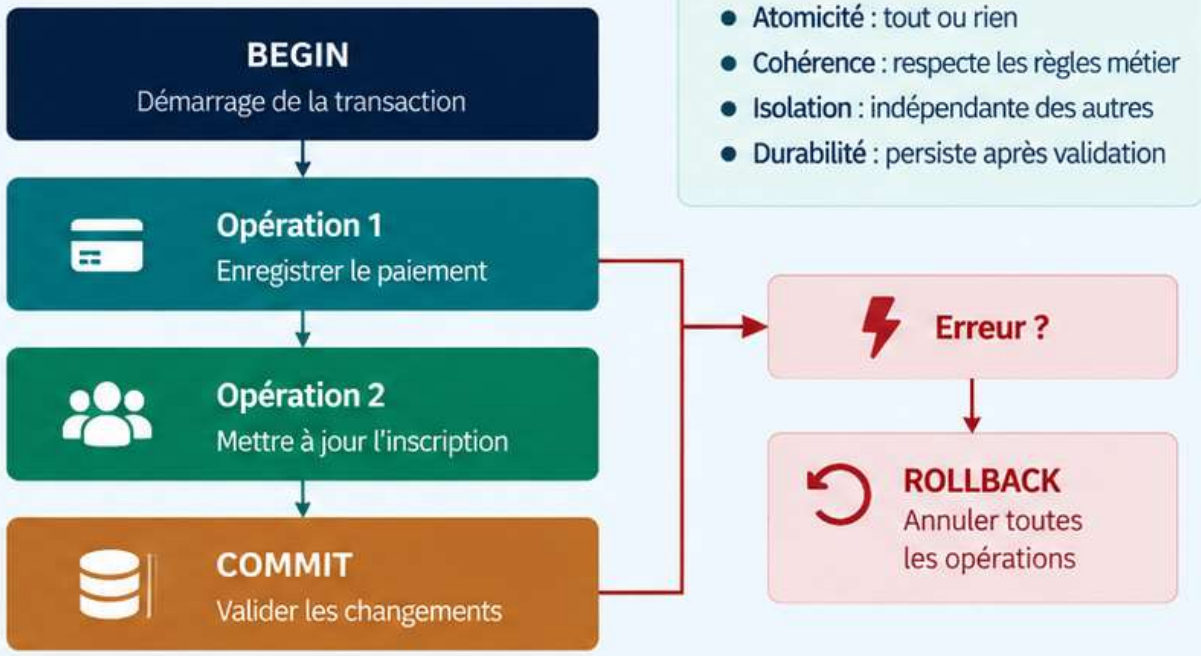
Unité logique
Une transaction regroupe plusieurs opérations comme une seule unité logique.

Inscription complète
Un paiement ne doit pas être enregistré sans mise à jour de l'inscription.

En cas d'échec
Si une opération échoue, l'ensemble des opérations est annulé (rollback).

Cohérence des données
La transaction protège la cohérence fonctionnelle du système.

Exemple : paiement d'une inscription
Deux opérations doivent réussir ensemble.



Propriétés ACID
- **Atomicité** : tout ou rien
- **Cohérence** : respecte les règles métier
- **Isolation** : indépendante des autres
- **Durabilité** : persiste après validation

À retenir
- Une transaction regroupe plusieurs opérations.
- En cas d'échec, toutes les opérations sont annulées.
- Elle garantit la cohérence des données.
- Indispensable pour les opérations métier critiques.

Bonnes pratiques
- Limiter la durée des transactions.
- Éviter les opérations externes (réseau, API) à l'intérieur.
- Gérer correctement les erreurs et les rollback.
- Tester les cas d'échec.

Concurrence

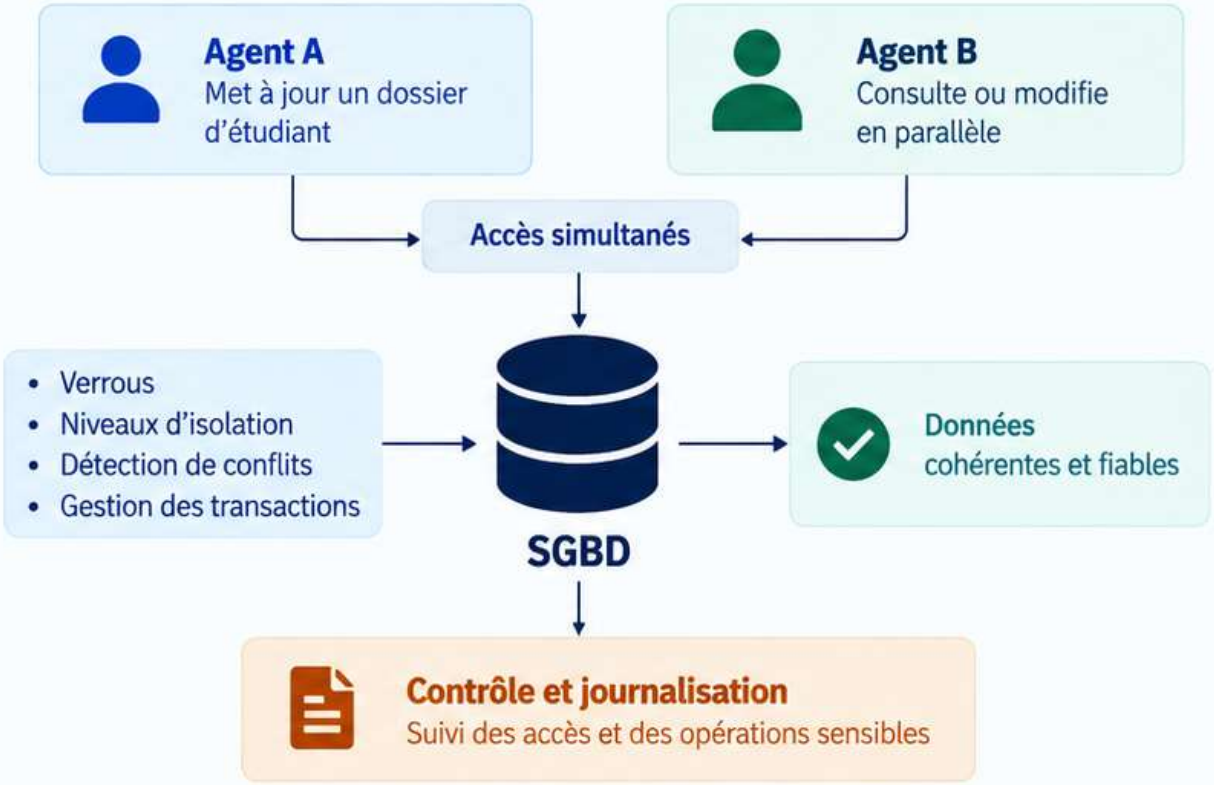
Gérer les accès simultanés sans compromettre la cohérence des données.

Accès simultanés
Plusieurs utilisateurs peuvent modifier des données au même moment.

Éviter les incohérences
Le SGBD doit garantir un état cohérent des données malgré les accès concurrents.

Verrous et niveaux d'isolation
Les verrous et niveaux d'isolation protègent les opérations sensibles.

Traitements critiques
Le concepteur doit identifier les traitements critiques et choisir la bonne stratégie de contrôle.



À retenir

- La concurrence est un enjeu réel dans les systèmes multi-utilisateurs.
- Le SGBD assure la cohérence grâce aux verrous et aux niveaux d'isolation.
- Il faut identifier les traitements critiques.
- Une bonne conception limite les conflits et améliore les performances.

Bonnes pratiques

- Choisir le bon niveau d'isolation.
- Limiter la durée des transactions.
- Éviter les verrous trop larges.
- Surveiller et analyser les conflits.
- Tester les scénarios d'accès simultanés.

PARTIE 3

Disponibilité et performance des données

Réplication, sauvegarde et cache

Comprendre le principe avant les outils

DISPONIBILITÉ

Réplication

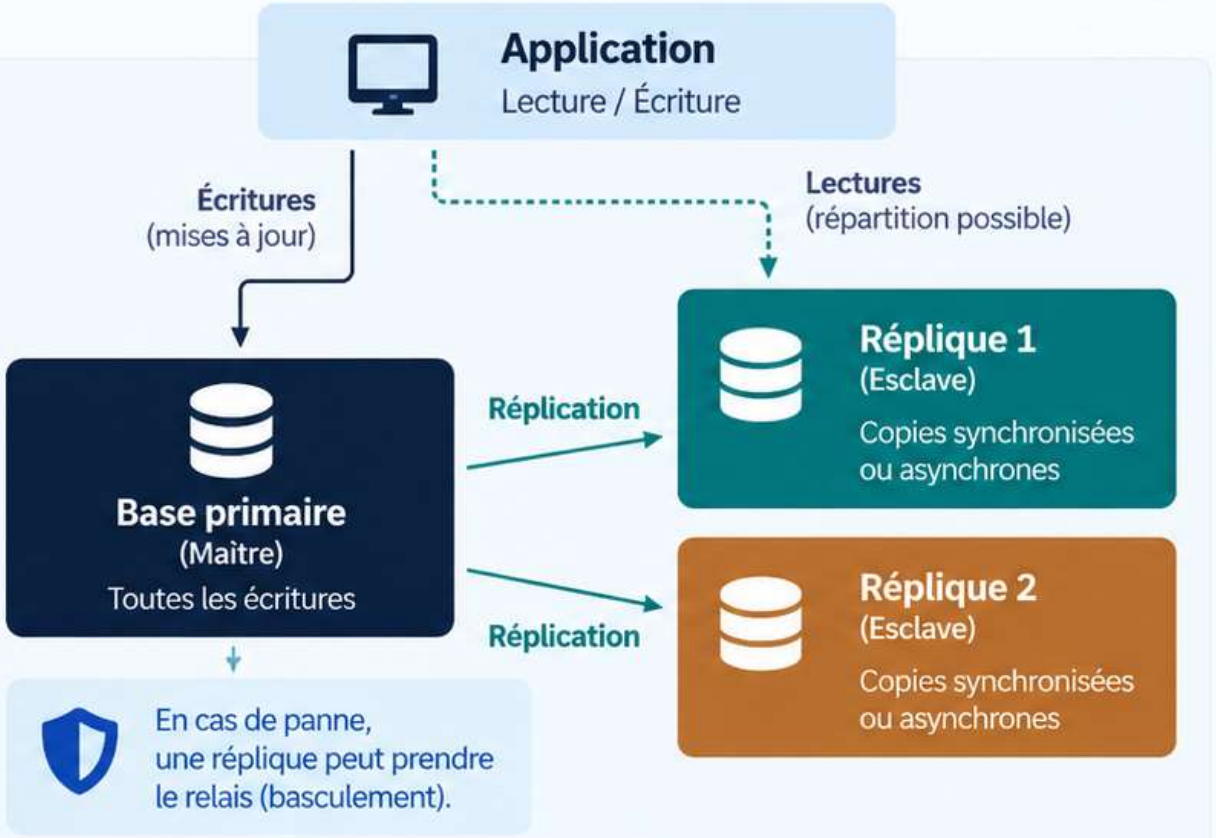
Assurer la disponibilité et la continuité de service.

Éviter le point unique de panne
Une base unique peut devenir un point unique de défaillance.

Une ou plusieurs copies
La réplication crée une ou plusieurs copies coordonnées des données.

Continuité de service
Elle améliore la disponibilité et permet de maintenir le service en cas de panne.

Répartition des lectures
Elle peut aussi répartir certaines lectures pour améliorer les performances.



À retenir

- La réplication augmente la disponibilité.
- Elle protège contre le point unique de panne.
- Elle peut améliorer les performances de lecture.
- Il existe plusieurs modes : synchrone, asynchrone, multi-région.

Bonnes pratiques

- Choisir le bon mode de réplication.
- Surveiller la latence et la cohérence.
- Tester la procédure de basculement.
- Distribuer les lectures sur les répliques.

COMPARER

Réplication synchrone et asynchrone

Deux approches pour assurer la disponibilité des données.

Synchrone

La réplication est confirmée avant de valider l'écriture.

- La primaire attend la réplique
- Cohérence renforcée
- Latence plus élevée
- Utile quand la perte est peu acceptable

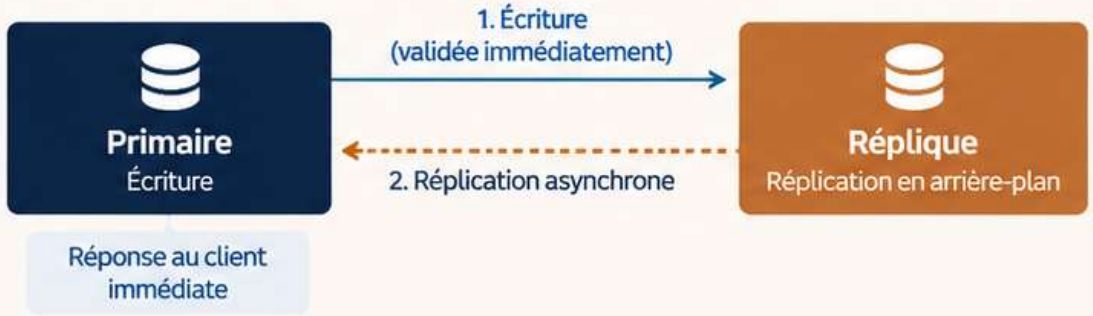


Garantie de cohérence maximale

Asynchrone

La réplication se fait après la validation de l'écriture.

- La primaire valide plus vite
- Meilleures performances
- Décalage possible
- Risque de perte très récente en panne brutale



Compromis performance / risque de perte

À retenir

- Synchrone : cohérence maximale, latence plus élevée.
- Asynchrone : meilleures performances, risque de décalage.
- Le choix dépend des exigences métier et du niveau de risque acceptable.

Bonnes pratiques

- Analyser les besoins de disponibilité.
- Évaluer la tolérance à la perte de données.
- Combiner les deux modes si nécessaire.
- Surveiller le décalage des répliques.

COMPARER

Réplication ou sauvegarde ?

Deux approches complémentaires pour protéger les données.

Réplication

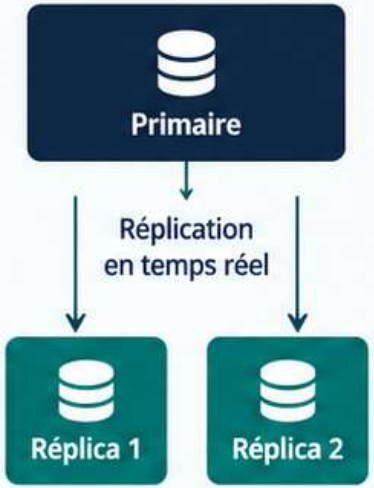
Assure la disponibilité des données en temps réel.

Objectif : disponibilité
Maintient le service même en cas de panne.

Copie rapidement les changements
Les modifications sont propagées en continu.

Protège contre certaines pannes
Permet un basculement automatique.

Peut propager une erreur humaine
Une suppression ou une mauvaise mise à jour est répliquée sur toutes les copies.



Idéal pour la haute disponibilité et la continuité de service.

Sauvegarde

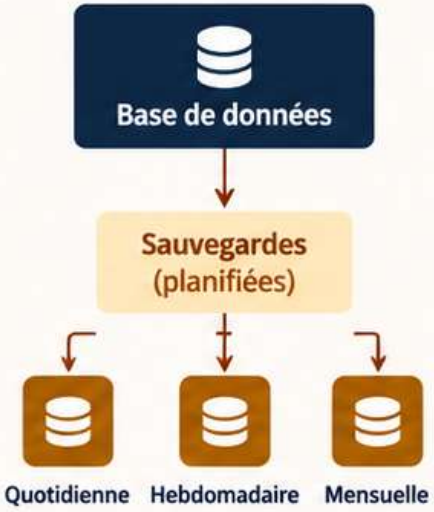
Permet de restaurer les données en cas de perte.

Objectif : restauration
Conserve un historique des données.

Conserve un historique
Garde des copies à différents moments dans le temps.

Permet de revenir dans le temps
Restaure un état antérieur des données.

Indispensable même avec réplication
Protège contre les erreurs humaines, les corruptions et les ransomwares.



Indispensable pour la récupération des données à long terme.

À retenir

- La réplication assure la disponibilité.
- La sauvegarde permet de restaurer les données.
- Les deux approches sont complémentaires.
- Le choix dépend des objectifs métier et du niveau de risque.

Bonnes pratiques

- Combiner réplication et sauvegarde.
- Tester régulièrement la restauration.
- Définir une politique de rétention.
- Automatiser et surveiller les sauvegardes.

Cache

Accélérer les accès aux données fréquemment demandées.

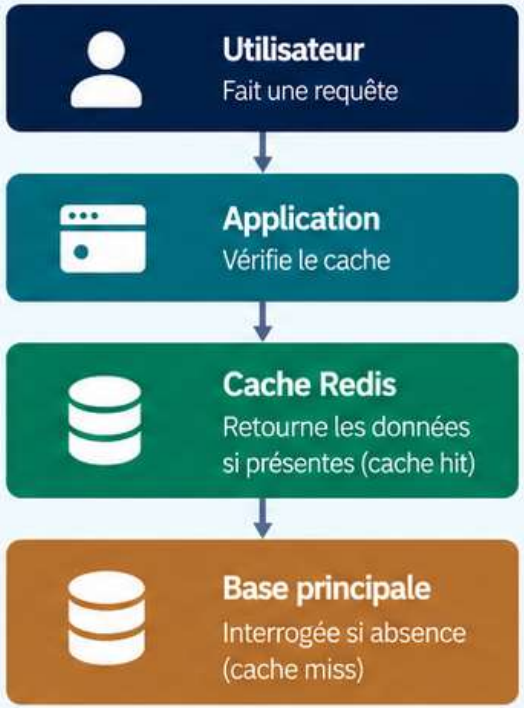
Données temporaires
Le cache conserve temporairement des données souvent demandées.

Moins de requêtes sur la base
Il évite d'interroger la base principale à chaque requête.

Meilleures performances
Il améliore la latence et absorbe les pics de lecture.

Ne remplace pas la base
Le cache ne remplace pas la base de référence, il la complète.

Flux de lecture avec cache



Avantages
- Réduit la charge sur la base
- Améliore la latence
- Absorbe les pics de lecture
- Améliore l'expérience utilisateur

Points d'attention
- Données potentiellement obsolètes
- Stratégie d'invalidation nécessaire
- Ne pas stocker de données sensibles
- Configurer la durée de vie (TTL)

À retenir
- Le cache stocke des données temporaires.
- Il réduit le nombre de requêtes sur la base principale.
- Il améliore les performances et la scalabilité.
- Il complète la base, sans la remplacer.

Bonnes pratiques
- Choisir les bonnes données à mettre en cache.
- Définir une stratégie d'invalidation (TTL, événements).
- Surveiller le taux de hit/miss.
- Adapter la taille du cache aux besoins.

COMPARER

Cache hit et cache miss

Deux scénarios pour comprendre le fonctionnement du cache.

Cache hit

La donnée est déjà en cache.

- La donnée est déjà en cache
- Réponse rapide
- La base est moins sollicitée
- Très utile pour données fréquentes

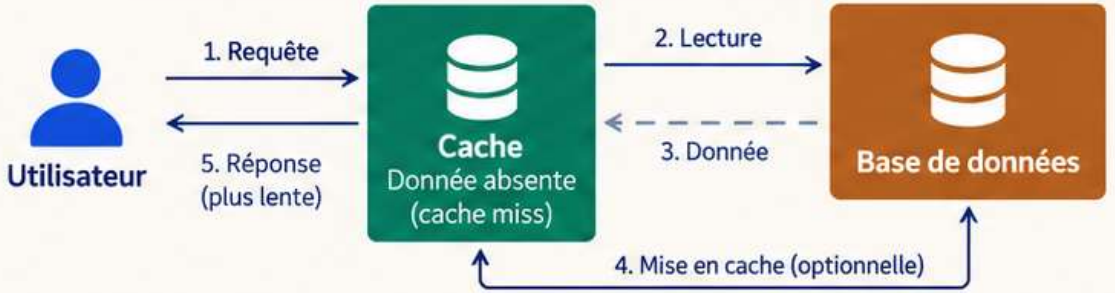


Accès rapide et moins de charge sur la base.

Cache miss

La donnée n'est pas en cache.

- La donnée est absente
- Lecture dans la base
- Mise en cache éventuelle
- Réponse un peu plus lente



Accès plus lent, mais la donnée peut être mise en cache.

À retenir

- Cache hit : réponse rapide depuis le cache.
- Cache miss : lecture dans la base, puis mise en cache.
- Un bon taux de hit améliore les performances.
- Le cache est particulièrement utile pour les données fréquentes.

Bonnes pratiques

- Choisir une bonne stratégie d'expiration (TTL).
- Surveiller le taux de hit/miss.
- Adapter la taille du cache aux besoins.
- Précharger les données critiques si nécessaire.

TTL et invalidation

Garder des données fiables dans le cache.

TTL (Time To Live)
Un TTL définit la durée de vie d'une donnée en cache.

Invalidation
L'invalidation supprime ou remplace une donnée devenue obsolète.

Risque d'incohérence
Une mauvaise invalidation peut afficher une ancienne information.

Données critiques
Les données critiques doivent être cachées avec prudence.

Cycle de vie d'une donnée en cache



À retenir

- Le TTL limite la durée de vie des données en cache.
- L'invalidation garantit la fraîcheur des données.
- Une mauvaise invalidation peut afficher des données obsolètes.
- Cacher les données critiques avec prudence.

Bonnes pratiques

- Choisir un TTL adapté au type de données.
- Mettre en place une stratégie d'invalidation.
- Surveiller la cohérence des données.
- Tester les scénarios de mise à jour.

SYNTHÈSE

Architecture données réaliste

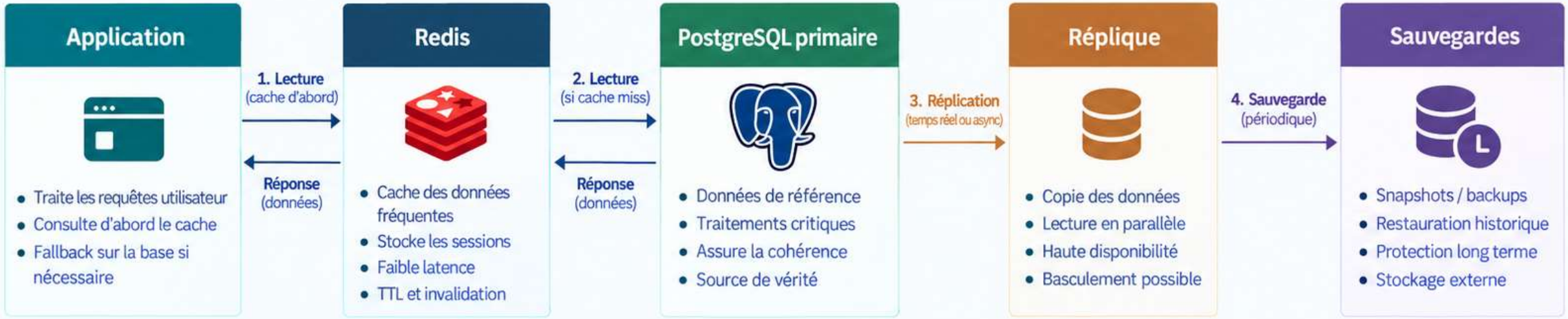
Une combinaison de technologies pour performance, disponibilité et restauration.

PostgreSQL
- Garde les données de référence
- Assure la cohérence et l'intégrité
- Source de vérité

Redis
- Accélère les lectures fréquentes
- Stocke les sessions et caches
- Réduit la charge sur PostgreSQL

Réplication
- Améliore la disponibilité
- Permet la lecture en parallèle
- Protège contre certaines pannes

Sauvegarde
- Permet la restauration historique
- Protège contre les erreurs humaines
- Conserve un historique des données



À retenir

- Chaque technologie a un rôle complémentaire.
- Le cache améliore les performances.
- La réplication garantit la disponibilité.
- La sauvegarde permet la récupération des données.

Bonnes pratiques

- Définir une stratégie de cache (TTL, invalidation).
- Surveiller la réplication et la latence.
- Tester régulièrement la restauration.
- Planifier la gestion de la capacité et la montée en charge.

ÉTUDE DE CAS

Cas : plateforme universitaire

Une plateforme pour gérer les données des étudiants, de la formation et de la vie universitaire.

Contexte

La plateforme gère étudiants, formations, inscriptions, paiements, factures et documents. Les données doivent rester cohérentes, disponibles et restaurables après incident.

Étudiants

Gestion des profils, inscriptions et dossiers.

Formations

Organisation des cursus et des modules.

Paiements & factures

Suivi des règlements et génération des factures.

Documents

Stockage et accès aux pièces justificatives.

Cohérence

Données fiables et à jour en permanence.

Restauration

Récupération rapide après incident.

Décisions à justifier

Pour garantir la fiabilité et la sécurité des données.

1. **Identifier les données relationnelles**
   Repérer les entités et leurs liens (ex. étudiant ↔ inscription).
2. **Déterminer ce qui peut être caché**
   Masquer les données sensibles (ex. NIR, coordonnées).
3. **Choisir une stratégie de transaction**
   Assurer l'intégrité des opérations (ex. ACID, verrouillage).
4. **Distinguer réplication et sauvegarde**
   Répliquer pour la disponibilité, sauvegarder pour la reprise.

À retenir

- Des données cohérentes et disponibles.
- Une gestion sécurisée des accès.
- Une stratégie de transaction adaptée.
- Une réplication et une sauvegarde complémentaires.

Bonnes pratiques

- Cartographier les données et leurs liens.
- Appliquer le principe du moindre accès.
- Tester les sauvegardes et la restauration.
- Surveiller et auditer régulièrement.

PARTIE 4

Réseaux

La communication entre composants d’architecture

Comprendre le principe avant les outils

CONTEXTE

Pourquoi le réseau dans l'architecture ?

Le réseau relie tous les composants et permet la communication.

Composants multiples

Le client, le serveur Web, l'API, la base de données et les services communiquent entre eux.

Transport des échanges

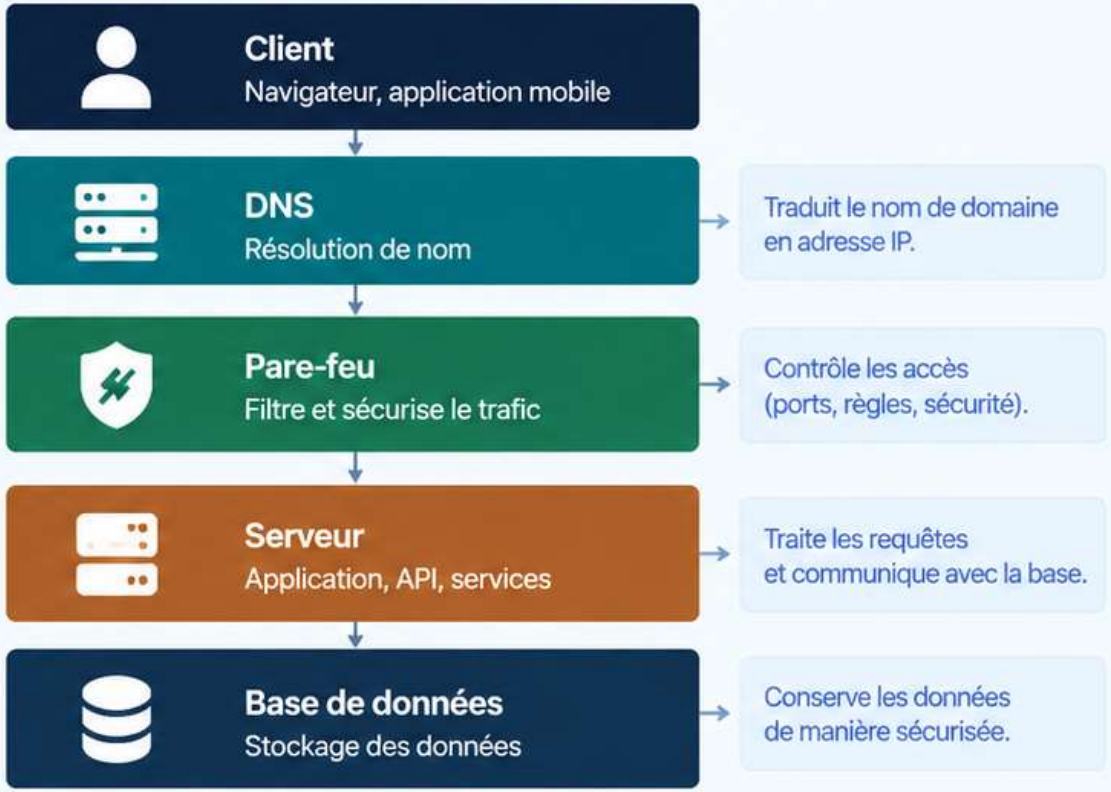
Le réseau transporte les requêtes, les réponses et les événements.

Impact sur la fiabilité

Une mauvaise conception réseau peut créer des latences, des indisponibilités ou des risques de sécurité.

Élément clé de l'architecture

Un réseau bien conçu isole les composants, contrôle les accès et améliore la résilience.



À retenir

- Le réseau relie tous les composants de l'application.
- Il transporte les requêtes, réponses et événements.
- Une mauvaise conception réseau fragilise l'architecture.
- La sécurité et l'isolation sont essentielles.

Bonnes pratiques

- Segmentation réseau et pare-feu.
- Contrôle des accès (principe du moindre privilège).
- Surveillance des performances (latence, disponibilité).
- Prévoir la tolérance aux pannes.

COMPARER

OSI et TCP/IP

Deux modèles complémentaires pour comprendre et utiliser les réseaux.

Modèle OSI

Un modèle de référence à 7 couches.

7. Application
6. Présentation
5. Session
4. Transport
3. Réseau
2. Liaison de données
1. Physique

7 couches conceptuelles
Organisation claire et normalisée.

Très utile pour raisonner
Permet d'analyser et diagnostiquer les problèmes réseau.

Sépare les fonctions
Couches physique, réseau, transport, application, etc.

Vision pédagogique détaillée
Idéal pour l'apprentissage et la compréhension.

Modèle TCP/IP

Un modèle opérationnel pour Internet.

- **Application** (HTTP, DNS, SMTP, etc.)
- **Transport** (TCP, UDP)
- **Réseau** (IP, ICMP, etc.)
- **Accès réseau** (Ethernet, Wi-Fi, etc.)

Modèle opérationnel
Utilisé dans Internet et les réseaux réels.

Couvre les besoins pratiques
Application, transport, réseau, accès.

Très lié aux protocoles réels
HTTP, TCP, IP, Ethernet, etc.

Vision plus pratique
Facilite la mise en œuvre et l'administration.

À retenir

- Le modèle OSI est un modèle de référence à 7 couches.
- Le modèle TCP/IP est un modèle opérationnel à 4 couches.
- Ils sont complémentaires et décrivent les mêmes fonctions.
- TCP/IP est le plus utilisé dans la pratique.

Cas d'usage

- Utiliser OSI pour comprendre et diagnostiquer.
- Utiliser TCP/IP pour concevoir et déployer.
- Connaître les protocoles associés à chaque couche.
- Savoir faire le lien entre les deux modèles.

Adresse IP et port

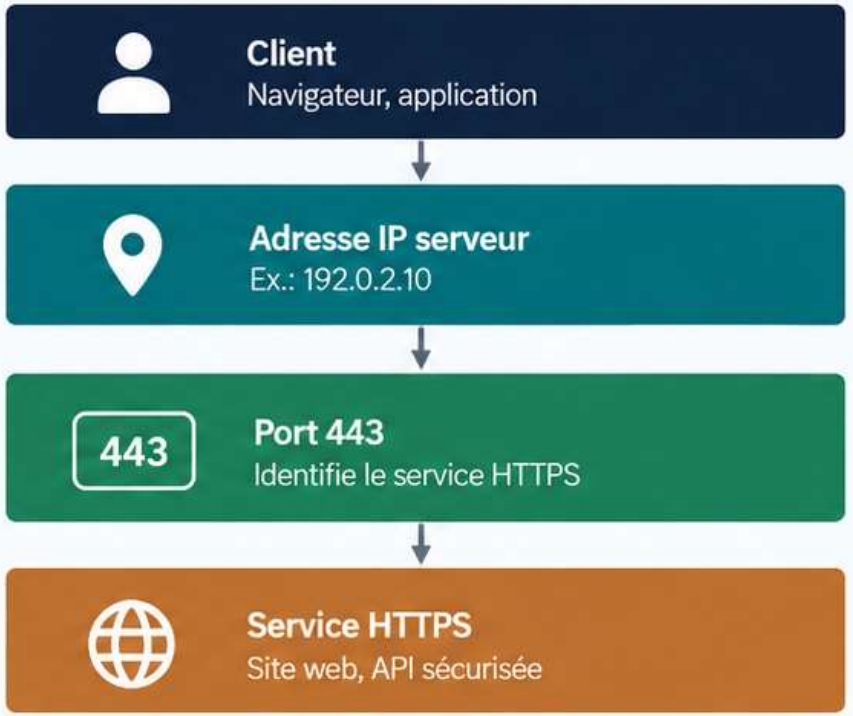
Deux informations complémentaires pour atteindre le bon service.

Adresse IP
L'adresse IP identifie une machine ou une interface réseau sur un réseau.

Port
Le port identifie le service applicatif sur cette machine.

Exemple HTTPS
HTTPS utilise généralement le port 443.

Complémentarité
La combinaison IP + port permet d'atteindre le bon service (la bonne application sur la bonne machine).



À retenir
- L'adresse IP identifie une machine.
- Le port identifie un service.
- HTTPS utilise le port 443.
- IP + port permet d'atteindre le bon service.

Bonnes pratiques
- Connaître les ports standards (ex. 80, 443).
- Utiliser des noms de domaine (DNS) plutôt que des IP.
- Sécuriser les services (HTTPS, pare-feu).
- Vérifier la connectivité (ping, telnet, curl).

NOM VERS ADRESSE

DNS

Le traducteur d'Internet.

Traduction de nom

Le DNS traduit un nom de domaine (ex. : portail.universite.sn) en adresse IP (ex. : 192.0.2.10).

Simplicité d'usage

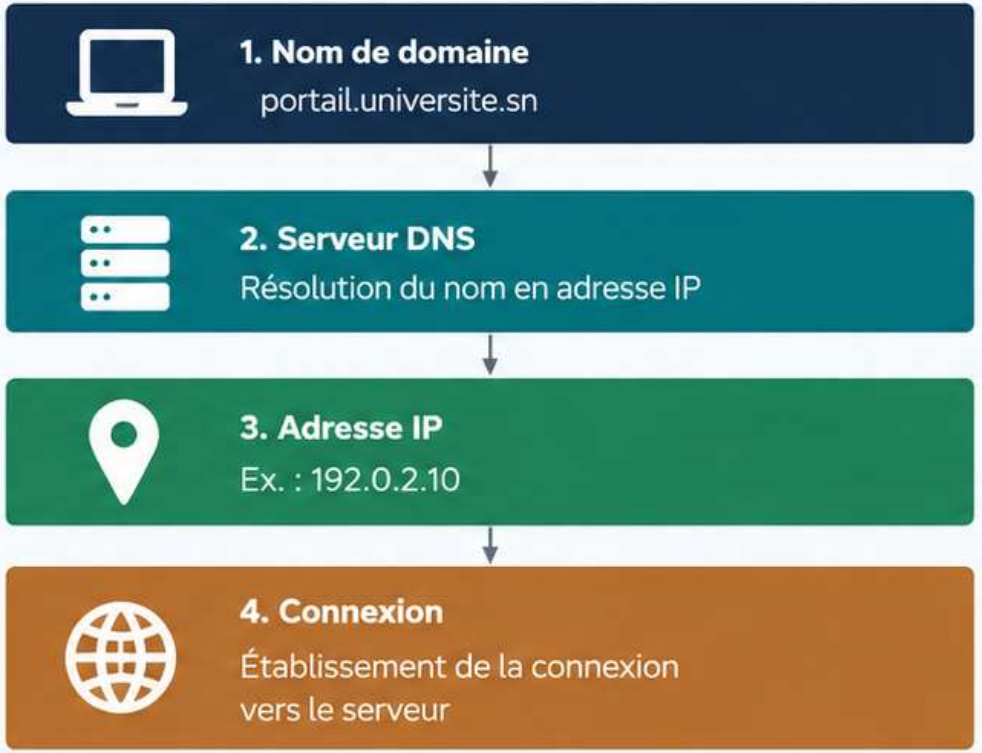
Il évite de retenir les adresses IP techniques et facilite l'accès aux services.

Intervention préalable

Le DNS intervient avant la connexion HTTP/HTTPS pour obtenir l'adresse IP du serveur.

Haute disponibilité

Une panne DNS peut rendre un service inaccessible même si le serveur fonctionne.



À retenir

- Le DNS traduit un nom en adresse IP.
- Il simplifie l'accès aux services.
- Il s'exécute avant la connexion HTTP/HTTPS.
- Une panne DNS peut bloquer l'accès.

Bonnes pratiques

- Utiliser des DNS fiables et redondants.
- Configurer un temps de cache adapté (TTL).
- Surveiller la disponibilité du service DNS.
- Prévoir un plan de secours (DNS secondaire).

COMPARER

HTTP et HTTPS

Deux protocoles pour l'échange de données sur le Web.

HTTP

Un protocole simple et non chiffré.

- Échange applicatif non chiffré
- Adapté aux environnements de test internes
- Port 80 par défaut
- À éviter pour les données sensibles



HTTPS

Un protocole sécurisé avec chiffrement.

- HTTP chiffré avec TLS
- Protège la confidentiallité et l'intégrité
- Port 443 par défaut
- Indispensable pour authentification et API



À retenir

- HTTP transmet les données en clair.
- HTTPS chiffre les données avec TLS.
- Ports par défaut : 80 (HTTP) et 443 (HTTPS).
- HTTPS est recommandé pour les sites web et les API.

Bonnes pratiques

- Utiliser HTTPS en production.
- Obtenir un certificat TLS valide.
- Rediriger automatiquement HTTP vers HTTPS.
- Vérifier la configuration des certificats.

FLUX APPLICATIF

Requête et réponse HTTP

Un échange structuré entre le client et le serveur.

Requête du client

Le client envoie une requête HTTP structurée (méthode, chemin, en-têtes, corps éventuel).

Traitement par le serveur

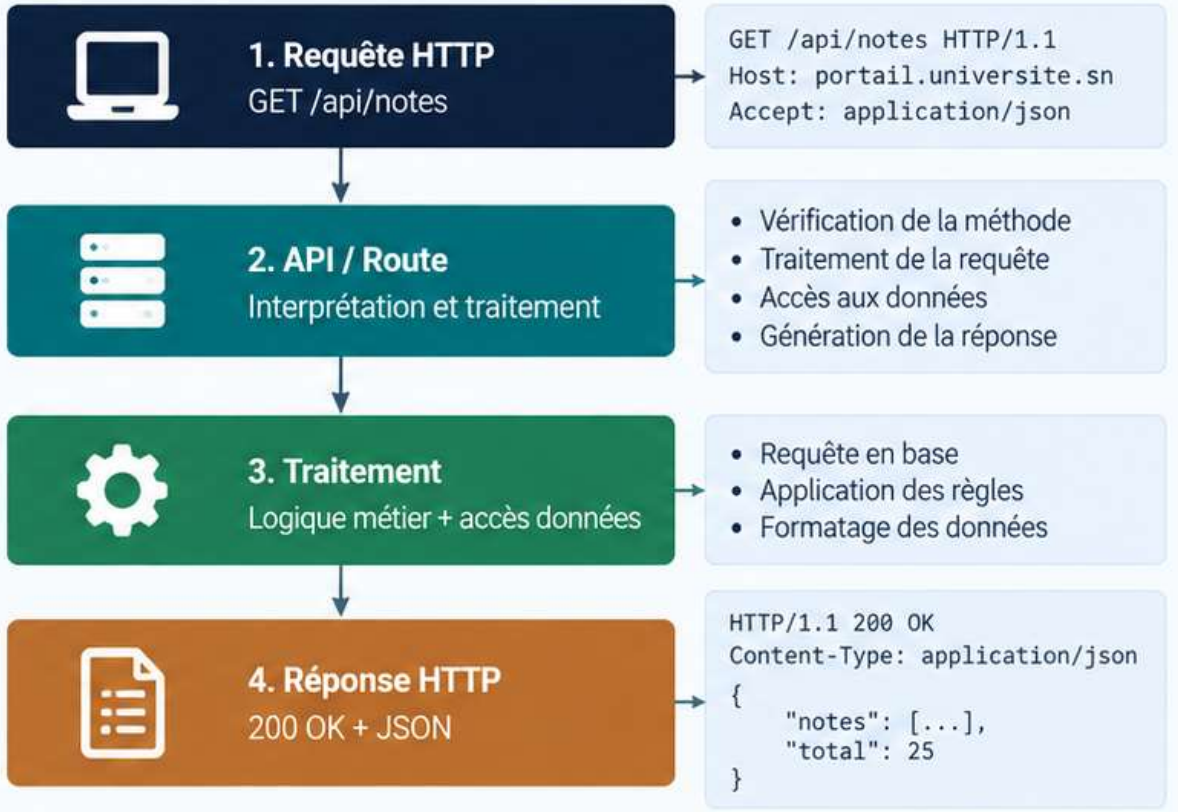
Le serveur interprète la méthode, le chemin, les en-têtes et parfois le corps.

Réponse du serveur

La réponse contient un code de statut, des en-têtes et souvent des données au format JSON ou HTML.

Base des API Web

Ces échanges sont la base des API Web et des applications modernes.



À retenir

- Le client envoie une requête structurée.
- Le serveur interprète la requête et traite la demande.
- La réponse contient un code, des en-têtes et des données.
- Ces échanges sont la base des API Web.

Bonnes pratiques

- Utiliser les méthodes HTTP adaptées (GET, POST, PUT, DELETE).
- Retourner des codes de statut appropriés (200, 400, 401, 500...).
- Utiliser le format JSON pour les API.
- Documenter les endpoints (ex. OpenAPI/Swagger).

SYNTHÈSE

Méthodes HTTP utiles

Les principales méthodes HTTP et leurs cas d'usage.

Méthode	Rôle courant	Exemple
GET	**Lire une ressource** Récupère des données sans les modifier.	`GET /api/etudiants/125`
POST	**Créer ou soumettre** Crée une nouvelle ressource.	`POST /api/paiements`
PUT	**Remplacer** Remplace complètement une ressource existante.	`PUT /api/dossiers/10`
PATCH	**Modifier partiellement** Met à jour seulement certains champs.	`PATCH /api/profil`
DELETE	**Supprimer** Supprime une ressource.	`DELETE /api/documents/9`

À retenir

- GET pour lire, POST pour créer.
- PUT pour remplacer, PATCH pour modifier partiellement.
- DELETE pour supprimer.
- Choisir la bonne méthode selon l'opération et l'API.

Bonnes pratiques

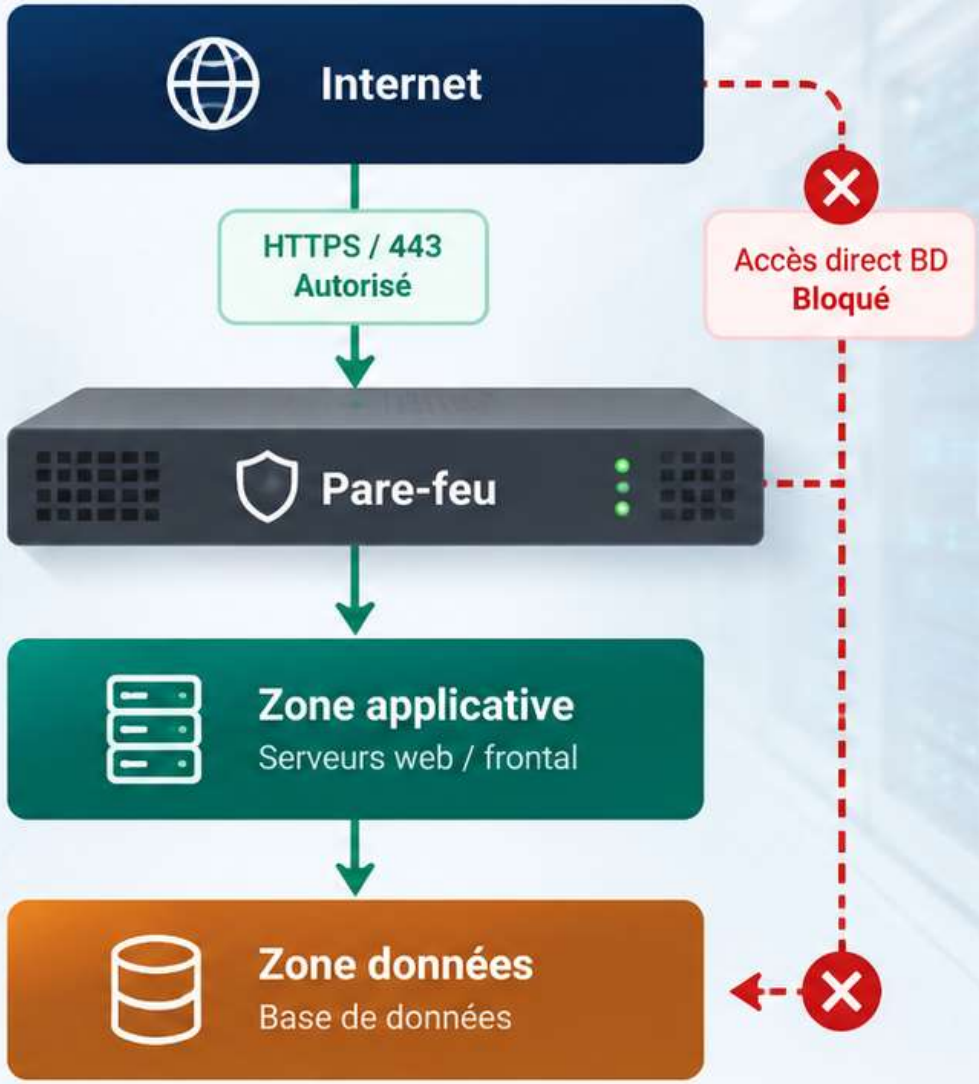
- Respecter la sémantique des méthodes.
- Utiliser les codes de statut HTTP appropriés.
- Documenter les endpoints de l'API.
- Tester les requêtes avec des outils (ex. curl, Postman).

Pare-feu

1. Le pare-feu filtre les flux entrants et sortants.
2. Il limite l'exposition des services sensibles.
3. Il peut autoriser HTTPS vers le frontal tout en bloquant la base depuis Internet.
4. Il matérialise une frontière de sécurité.

À retenir

Le pare-feu applique des règles de filtrage pour n'autoriser que les flux nécessaires et protéger les ressources internes.



RÉDUIRE L'EXPOSITION

Segmentation réseau

Des zones isolées pour plus de sécurité.

Isolation des zones

La segmentation sépare les zones selon leur criticité (Internet, DMZ, applications, données, administration).

Accès restreint

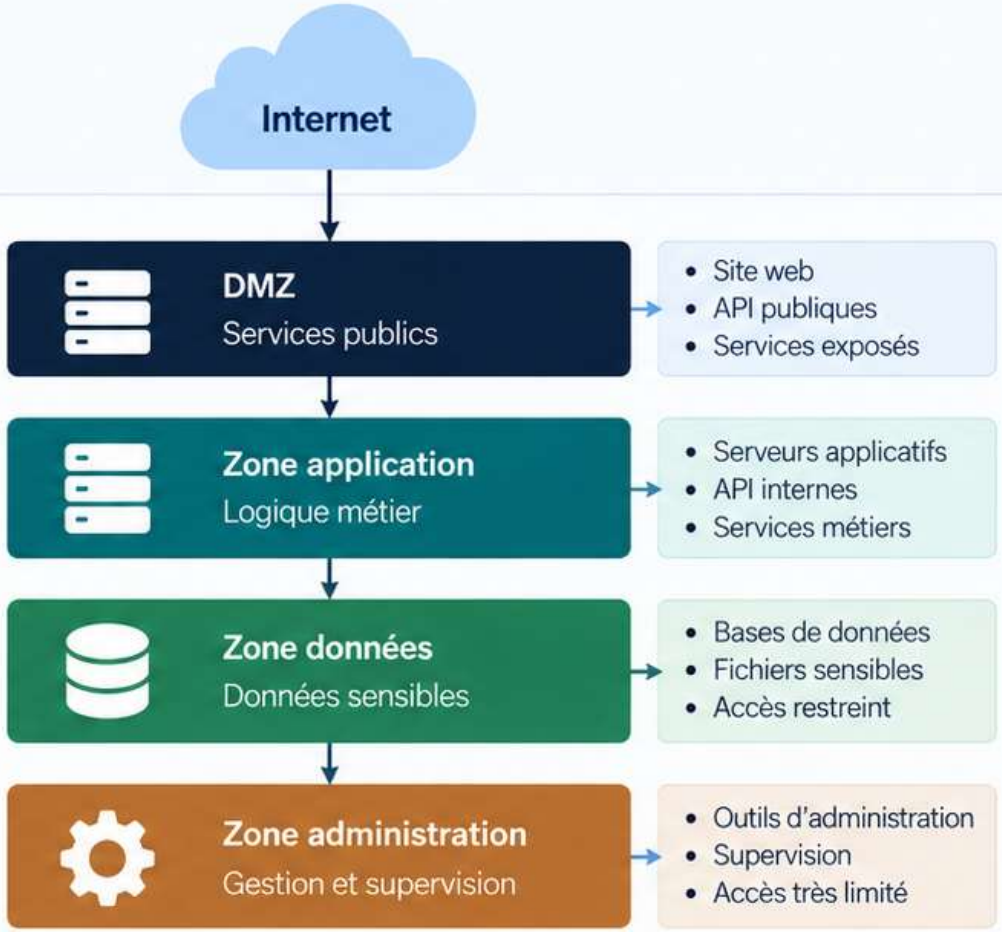
Les utilisateurs ne doivent pas atteindre directement les bases de données.

Contrôle des flux

Les flux entre zones doivent être explicitement autorisés par des règles (pare-feu).

Réduction des risques

Elle limite l'impact d'une compromission en confinant l'attaque à une zone.



À retenir

- Séparer les zones selon leur criticité.
- Restreindre les accès entre zones.
- Autoriser uniquement les flux nécessaires.
- Limiter l'impact d'une compromission.

Bonnes pratiques

- Appliquer le principe du moindre privilège.
- Utiliser des pare-feu entre les zones.
- Surveiller et journaliser les flux.
- Tester la segmentation régulièrement.

RÉPARTIR LA CHARGE

Load balancing

Distribuer les requêtes pour plus de performance et de disponibilité.

Répartition des requêtes

Le load balancer distribue les requêtes vers plusieurs serveurs applicatifs selon une stratégie (round-robin, poids, etc.).

Amélioration de la disponibilité

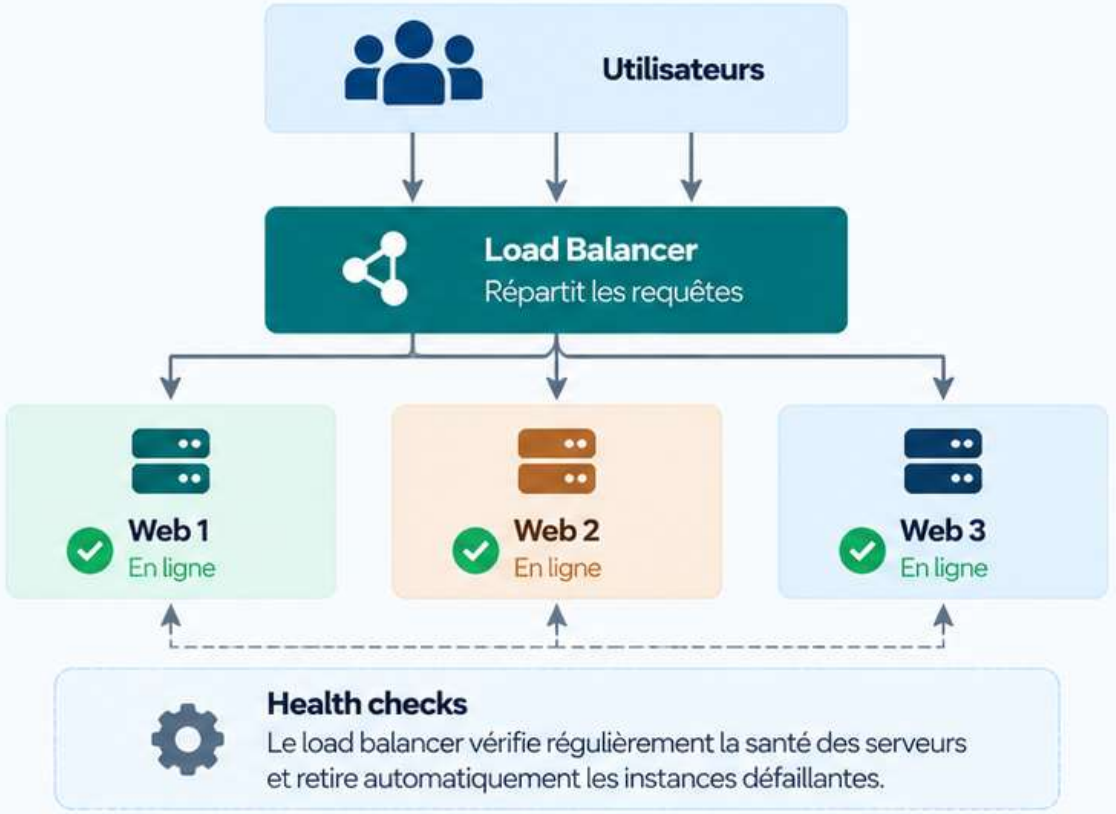
Si une instance tombe, les requêtes sont redirigées vers les instances saines.

Montée en charge

Il permet d'ajouter facilement des serveurs pour gérer plus de trafic.

Supervision

Il vérifie l'état de santé des serveurs (health checks) pour ne diriger le trafic que vers les instances disponibles.



À retenir

- Distribue les requêtes vers plusieurs serveurs.
- Améliore la disponibilité et la résilience.
- Permet la montée en charge horizontale.
- Surveille l'état de santé des serveurs.

Bonnes pratiques

- Utiliser des health checks.
- Choisir une stratégie de répartition adaptée.
- Prévoir plusieurs instances dans des zones différentes.
- Surveiller les métriques (latence, taux d'erreur, charge).

COMPARER

Load balancing : deux objectifs

Performance et disponibilité pour des services plus fiables.

Performance

Répondre efficacement à la demande.

- Répartir les requêtes
- Absorber davantage d'utilisateurs
- Éviter la saturation d'un seul serveur
- Permettre l'élasticité

Résultat

Des temps de réponse plus courts
et une meilleure expérience utilisateur.

Disponibilité

Assurer un service continu.

- Détecter une instance indisponible
- Retirer un serveur défaillant
- Maintenir le service
- Faciliter les mises à jour progressives

Résultat

Un service plus résilient et disponible
même en cas de panne.

À retenir

- Le load balancing améliore à la fois la performance et la disponibilité.
- Il répartit les requêtes vers plusieurs serveurs.
- Il détecte et retire automatiquement les serveurs défaillants.
- Il permet de faire face à la croissance du trafic.

Bonnes pratiques

- Configurer des health checks.
- Utiliser une stratégie de répartition adaptée.
- Prévoir plusieurs instances dans des zones différentes.
- Surveiller les métriques et les logs.

SYNTHÈSE

Architecture réseau applicative

Une chaîne de composants pour un service sécurisé, performant et disponible.

- Le DNS dirige vers l'entrée publique.
- Le pare-feu contrôle l'accès.
- Le load balancer répartit la charge.
- Les serveurs applicatifs accèdent aux données selon des flux internes maîtrisés.



À retenir

- Chaque composant a un rôle spécifique.
- La sécurité est assurée par le pare-feu et la segmentation.
- La performance est apportée par le load balancing.
- L'ensemble garantit un service fiable et scalable.

Bonnes pratiques

- Utiliser HTTPS et des règles de pare-feu strictes.
- Mettre en place des health checks.
- Segmenter les réseaux (DMZ, applicatif, données).
- Surveiller et journaliser les flux.